These problems cover the monad material of Lecture 14. Throughout, `let*` is the monadic bind from the lectures, and the following definitions are assumed.

Option monad.

```
let return v = Some v
let bind m f = match m with None -> None | Some v -> f v
let ( let* ) = bind
let ( >>= ) = bind
```

State monad. State (S : **sig type** t **end**) : STATE **with type** state = S.t, implemented as

```
type 'a t = state -> state * 'a
let return v   = fun s -> (s, v)
let (let*) m f = fun s -> let (s', a) = m s in f a s'
let get        = fun s -> (s, s)
let put s'     = fun _ -> (s', ())
let run_state m init = m init
```

In the state questions, IntState = State (**struct type** t = int **end**) has been opened, and

```
let inc v  = let* s = get in put (s + v)
let dec v  = let* s = get in put (s - v)
let double = let* s = get in put (s * 2)
```

Writer monad. WRITER : WRITERMONAD **with type** 'a t = string * 'a, implemented as

```
let return v = ("", v)
let (let*) (s, v) f = let (s', v') = f v in (s ^ s', v')
let printline (s : string) = (s, ())
```

together with the lecture's inc x = x + 1, dec x = x − 1 and

```
let log name f x = ("Called " ^ name ^ " on " ^ string_of_int x ^ ";", f x)
```

Part A. The option monad

1. Evaluate each expression by hand.
   (a) `let* x = Some 4 in let* y = Some 6 in return (x * y)`
   (b) `let* x = Some 4 in let* y = None in return (x * y)`
   (c) `let* x = None in let* y = Some 6 in return (x * y)`
   (d) `let* x = Some 4 in if x > 3 then None else return x`
   (e) `let* x = Some 2 in if x > 3 then None else return x`
   (f) `bind (Some 5) return`
   (g) `Some 5 >>= fun x -> Some (x - 1)>>= fun y -> return (x + y)`
2. For each expression, say whether it typechecks. If it does, give its value and type. If it does not, say which part has the wrong type.
   (a) `let* x = Some 3 in x`
   (b) `let* x = 3 in return x`
   (c) `let* x = Some 3 in Some x`
   (d) `let* x = Some (Some 3)in x`
   (e) `return (let* x = Some 3 in return x)`
   (f) `let* x = Some 3 in let y = x + 1 in return y`
   (g) `let* x = Some "a"in return (x ^ "b")`

3. Recall the lecture's interpreter in the option monad:

```
type expr = Val of int | Plus of expr * expr | Div of expr * expr

let rec eval e = match e with
  | Val v -> return v
  | Plus (e1, e2) ->
      let* v1 = eval e1 in
      let* v2 = eval e2 in
      return (v1 + v2)
  | Div (e1, e2) ->
      let* v1 = eval e1 in
      let* v2 = eval e2 in
      if v2 = 0 then None else return (v1 / v2)
```

   (a) Extend expr with Minus **of** expr * expr and Mod **of** expr * expr (note that remainder by zero is a failure, just like division), and extend eval.

   (b) Suppose the Val case is changed to announce itself:

```
| Val v -> print_endline ("Visiting Val " ^ string_of_int v); return v
```

   List exactly the lines printed, and the value returned, by

   (i) eval (Plus (Div (Val 6, Val 0), Plus (Val 1, Val 2)))

   (ii) eval (Plus (Plus (Val 1, Val 2), Div (Val 6, Val 0)))

   (c) What are eval (Mod (Val 7, Minus (Val 2, Val 2))), eval (Mod (Val 7, Val 3)) and eval (Minus (Val 2, Div (Val 9, Val 2)))?

4. Consider

```
let* x = Some 1 in
let* y = (print_endline "A"; None) in
let* z = (print_endline "B"; Some 2) in
return (x + y + z)
```

   (a) What is printed, and what is the value of the expression?

   (b) Same questions if None on the second line is replaced by Some 10.

Part B. The state monad

5. Evaluate each of the following.

   (a) run_state (**let*** _ = put 3 **in** **let*** _ = put 4 **in** get) 0

   (b) run_state (**let*** a = get **in** **let*** _ = put (a + 10) **in** **let*** b = get **in** return (b − a)) 5

   (c) run_state (**let*** a = get **in** **let*** _ = put 0 **in** **let*** _ = put a **in** get) 7

   (d) run_state (**let*** _ = inc 5 **in** **let*** _ = inc 5 **in** **let*** s = get **in** return (s, s)) 0

   (e) run_state (**let*** x = return 3 **in** **let*** y = get **in** return (x + y)) 4

   (f) run_state (**let*** _ = double **in** **let*** _ = inc 1 **in** **let*** _ = double **in** get) 1

   (g) With **let** c = inc 1: run_state (**let*** _ = c **in** **let*** _ = c **in** get) 0 and (run_state c 0, run_state c 0).

6. With IntState opened, which of the following typecheck? Give the value (or type) of those that do, and the reason for those that do not.
   (a) `get 5`
   (b) `let* s = get in s`
   (c) `run_state (let* s = get in put s) 3`
   (d) `run_state get 9`
   (e) `let* () = get in return 1`
   (f) `let* _ = put 5 in put "x"`
   (g) `put`

7. Write the following computations in IntState.
   (a) Define `next : int t`, a computation that returns the current state and increments it, and then `label : 'a list -> (int * 'a) list t`, which pairs every element of the list with a fresh number obtained from next, in order. For example, `run_state (label ["a"; "b"; "c"]) 0` should produce the result `(3, [(0, "a"); (1, "b"); (2, "c")])`.
   (b) `fetch_add : int -> int t`, which yields the current state and adds `n` to it, so that `run_state (fetch_add 5) 10 = (15, 10)`.
   (c) `sum_list : int list -> unit t`, which adds every element of the list to the state, so that `run_state (sum_list [1; 2; 3]) 10 = (16, ())`.
   (d) `run_twice : 'a t -> ('a * 'a) t`, which runs a computation twice and pairs the two values, so that `run_state (run_twice next) 0 = (2, (0, 1))`.
   (e) `max_state : int list -> unit t`, which replaces the state by the maximum of the state and all the elements, so that `run_state (max_state [3; 9; 2]) 5 = (9, ())` and `run_state (max_state []) 5 = (5, ())`.

Part C. The writer monad

8. With WRITER opened:
   (a) Evaluate
      (i) `let* () = printline "a" in let* () = printline "b" in let* () = printline "c" in return 0`
      (ii) `let* x = ("x;", 1) in let* y = ("y;", 2) in return (x + y)`
      (iii) `let* x = return 1 in let* () = printline "hi" in return (x + 1)`
      (iv) `let* () = printline "B" in let* () = printline "A" in return ()`
   (b) Define `printlines : string list -> unit t` so that `printlines ["a"; "b"; "c"] = ("abc", ())`.
   (c) Define `countdown : int -> unit t` so that `countdown 3 = ("3,2,1,", ())`.
   (d) Define a writer monad whose log is a *list* of strings, `type 'a t = string list * 'a`, with `return`, `(let*)` and `tell : string -> unit t`, so that `let* () = tell "a" in let* () = tell "b" in return 5` is `(["a"; "b"], 5)`.